FATEC COTIA · ENGENHARIA DE SOFTWARE II

IFO — Organizador Financeiro Inteligente

Documentação do projeto · Versão 2 (refatoração)
Escopo, metodologia, requisitos, modelo de dados, diagramas e roadmap

Equipe
João Marcos Vilaça Marques · Amanda Freitas de Lima
Lucas de Souza Saldanha dos Santos · Vinicius de Amorim Farias

Curso Desenvolvimento de Software Multiplataforma
Disciplina Engenharia de Software II
Período 2º Semestre

Sumário

01 · Escopo do Projeto
1. Visão
2. Objetivos
3. Dentro do escopo (V2 atual)
4. Fora do escopo (decisões já tomadas)
5. Decisões recentes de escopo
6. Premissas e restrições
7. Partes interessadas
8. Critérios de sucesso
9. Riscos

02 · Requisitos e Regras de Negócio
1. Requisitos funcionais
2. Requisitos não funcionais
3. Regras de negócio
4. Categorias fixas
5. Rastreabilidade resumida

03 · Metodologia de Desenvolvimento
1. Abordagem
2. Fluxo de trabalho com Git
3. Definition of Ready (pronto para começar)
4. Definition of Done (pronto para entregar)
5. Qualidade e testes
6. Arquitetura e padrões
7. Documentação como código

04 · Modelo de Dados
1. Decisões do modelo
2. Diagrama entidade-relacionamento
3. Dicionário de dados
4. Migration alvo — `V1__criar_esquema.sql`
5. Migration de dados — `V2__categorias_fixas.sql`
6. Mapeamento JPA (orientação)

05 · Diagramas
1. Diagrama de casos de uso
2. Diagrama de classes
3. Diagramas de sequência
4. Diagramas de processo (BPMN simplificado)
5. Arquitetura em camadas

06 · Contrato da API (alvo V2)
1. Autenticação (público)
2. Usuário
3. Categorias (somente leitura)
4. Transações
5. Orçamento
6. Metas
7. Calculadora de investimentos (sem persistência)
8. Formato de erro (todas as rotas)
9. Despesas recorrentes
10. Exportação CSV

07 · Roadmap, Backlog e Decisões
1. Visão geral das sprints
2. Backlog detalhado
3. Checklist de bugs visuais (preencher na R4)
4. Backlog IFO V2 (futuro, fora deste ciclo)
5. Decisões de arquitetura (ADRs)

08 · Diagnóstico da V1
1. Segurança
2. Banco de dados e modelo
3. Regras de negócio
4. API e backend
5. Frontend
6. Testes
7. Divergências entre o PDF do seminário e o código

01 · Escopo do Projeto

Projeto: IFO — Organizador Financeiro Inteligente (versão 2, “IFO V1 → refatoração”) **Equipe:** João Marcos Vilaça Marques, Amanda Freitas de Lima, Lucas de Souza Saldanha dos Santos, Vinicius de Amorim Farias **Instituição:** FATEC Cotia — Desenvolvimento de Software Multiplataforma (Engenharia de Software II)

---

1. Visão

O IFO é uma aplicação web que ajuda pessoas a controlar as finanças pessoais de forma simples: registrar receitas e despesas, seguir a regra 50/30/20, acompanhar metas e simular investimentos.

Problema: grande parte da população tem dificuldade para controlar gastos, o que leva a endividamento e falta de planejamento. Ferramentas existentes costumam ser complexas demais para quem está começando.

Proposta: um sistema leve, em português, com poucas telas e regras claras, que orienta o usuário pela divisão da renda em **Necessidades (50%)**, **Lazer (30%)** e **Investimentos (20%)** e avisa quando ele estoura um limite.

2. Objetivos

Geral: entregar um sistema web de gestão financeira pessoal com qualidade de engenharia (arquitetura em camadas, segurança, testes e documentação).

Específicos

1. Corrigir os problemas estruturais da V1 (segurança, inconsistência entre banco e código, regras faltantes).
2. Reduzir o escopo ao que realmente é usado (remover Conta/Carteira e persistência de Investimento).
3. Implementar o modelo **Grupo → Categoria (fixa) → Descrição (livre)** nas transações.
4. Permitir que o usuário **defina o próprio orçamento** (padrão 50/30/20, editável) e receba **alerta simples de estouro**.
5. Entregar interface **responsiva (mobile-first)** com design renovado.
6. Manter a documentação (UML, BPMN, requisitos) sincronizada com o código.

3. Dentro do escopo (V2 atual)

#	Funcionalidade	Resumo
1	Cadastro e login	Autenticação segura (BCrypt + JWT)
2	Perfil	Editar nome e renda mensal
3	Transações	CRUD de receitas e despesas com categoria fixa + descrição livre
4	Filtro por mês	Listagem e totais por mês/ano
5	Resumo financeiro	Saldo, total de receitas e total de despesas do mês
6	Orçamento 50/30/20	Percentuais por grupo, editáveis pelo usuário (soma = 100%)
7	Alerta de estouro	Aviso simples em 80% (atenção) e acima de 100% (estouro)
8	Metas financeiras	CRUD, aporte manual e barra de progresso
9	Calculadora de investimentos	Juros simples e compostos, **sem salvar** no banco
10	Gráfico de pizza simples	Distribuição das despesas do mês por grupo
11	Despesas recorrentes	Modelo mensal (ex.: internet, luz) que gera a transação automaticamente
12	Exportar CSV	Download das transações do mês filtrado
13	Dark mode	Tema claro/escuro alternável, com padrão do sistema

4. Fora do escopo (decisões já tomadas)

- **Conta/Carteira** (tabela, API e diagramas) — removida; não será usada.
- **Persistência de simulações de investimento** — a calculadora é apenas cálculo.
- **Categorias criadas pelo usuário** — a lista de categorias é fixa; o usuário altera apenas a **descrição** da transação.
- **Dashboards interativos** (gráficos dinâmicos, cartões avançados, drill-down) — ficam para o **IFO V2 (futuro)**.
- Integração bancária, app mobile nativo, 2FA e IA para sugestões — backlog futuro.

5. Decisões recentes de escopo

Em **18/10/2026** a equipe aprovou a inclusão de **despesas recorrentes**, **exportar CSV** e **dark mode** (prioridade *Should*: entram depois dos itens *Must*). Os detalhes estão em `02-requisitos.md` (RF20–RF22, RN14–RN19) e no roadmap (`07-roadmap.md`).

> Regra: qualquer item novo só entra na sprint com decisão registrada em `docs/` (ver ADRs no fim do `07-roadmap.md`).

6. Premissas e restrições

- Usuário único por conta (sem compartilhamento entre pessoas).
- Moeda: Real (BRL). Datas no formato brasileiro na interface.
- Stack mantida: Java 17 + Spring Boot 3.2, PostgreSQL 15, HTML/CSS/JavaScript puro, Docker para o banco.
- Equipe de 4 pessoas, sprints semanais, prazo acadêmico.
- O frontend **não** usa frameworks pesados (decisão da V1 mantida para leveza e controle).

7. Partes interessadas

Papel	Quem	Interesse
Usuário final	Pessoas que querem organizar o dinheiro	Simplicidade e clareza
Equipe de desenvolvimento	4 alunos	Aprendizado e entrega
Professor(a) de Engenharia de Software II	FATEC Cotia	Aplicação de métodos e documentação

8. Critérios de sucesso

- Todos os requisitos “Must” de `02-requisitos.md` implementados e testados.
- Nenhuma senha em texto puro; nenhum endpoint expõe dados de outro usuário.
- Banco criado do zero apenas por migrations (sem scripts manuais) e aplicação sobe com um comando.
- Telas utilizáveis de 360 px (celular) a desktop.
- Documentação e diagramas coerentes com o código entregue.

9. Riscos

Risco	Impacto	Mitigação
Divergência código × documentação (como na V1)	Alto	Diagramas em Mermaid dentro do repo, revisados no Definition of Done
Escopo crescer (V2 invadir a V1)	Médio	Lista “fora do escopo” + ADRs
Tempo curto	Médio	Priorização MoSCoW e sprints curtas
Problemas de ambiente (Docker/CORS)	Baixo	`docker-compose` + variáveis de ambiente documentadas

02 · Requisitos e Regras de Negócio

Prioridade segue MoSCoW: **M** (Must), **S** (Should), **C** (Could).

1. Requisitos funcionais

Conta do usuário

ID	Requisito	Prior.
RF01	O usuário se cadastra informando nome, e-mail, senha e renda mensal.	M
RF02	O usuário faz login com e-mail e senha e recebe um token JWT.	M
RF03	O usuário encerra a sessão (logout) no cliente.	M
RF04	O usuário edita nome e renda mensal.	M

Transações

ID	Requisito	Prior.
RF05	O usuário cadastra transação informando valor, data, **categoria** (lista fixa) e **descrição** livre. O tipo (receita/despesa) e o grupo são derivados da categoria.	M
RF06	O usuário lista, edita e exclui suas transações. A edição é um `PUT` (não “apagar e recriar”).	M
RF07	O usuário filtra transações e totais por **mês/ano**.	M
RF08	O sistema exibe o resumo do mês: total de receitas, total de despesas e saldo.	M
RF09	O sistema lista as categorias disponíveis (somente leitura), agrupadas.	M
RF10	O sistema exibe **gráfico de pizza simples** das despesas do mês por grupo.	S

Orçamento

ID	Requisito	Prior.
RF11	Ao se cadastrar, o usuário recebe o orçamento padrão 50/30/20.	M
RF12	O usuário edita os percentuais de Necessidades, Lazer e Investimentos; a soma deve ser 100%.	M
RF13	O sistema calcula, por grupo, limite (R$), gasto no mês, % usado e nível (OK / ATENÇÃO / ESTOURO).	M
RF14	Ao lançar uma despesa, o sistema avisa se o grupo entrou em ATENÇÃO ou ESTOURO. O aviso também aparece no dashboard.	M

Metas

ID	Requisito	Prior.
RF15	O usuário cria, lista, edita e exclui metas (título, valor objetivo, prazo).	M

RF16	O usuário registra aportes na meta (atualiza o valor atual).	M
RF17	O sistema exibe barra de progresso (%) e marca a meta como concluída ao atingir o objetivo.	M

Calculadora de investimentos

ID	Requisito	Prior.
RF18	O usuário informa aplicação inicial (opcional), aporte mensal, taxa mensal (%), prazo (meses) e tipo de juros (simples/composto) e recebe montante final, total investido e rendimento.	M
RF19	O resultado traz a evolução mês a mês em tabela. **Nada é salvo no banco.**	S

Recursos complementares (aprovados em 18/10/2026)

ID	Requisito	Prior.
RF20	**Despesas recorrentes:** o usuário cadastra, lista, edita, pausa e exclui um modelo mensal (categoria de despesa, valor, descrição, dia do mês, início e fim opcional). O sistema gera a transação automaticamente a cada mês.	S
RF21	**Exportar CSV:** o usuário baixa as transações do mês/ano filtrado em arquivo CSV.	S
RF22	**Dark mode:** o usuário alterna entre tema claro e escuro; o padrão segue o sistema e a escolha fica salva no navegador.	S

2. Requisitos não funcionais

ID	Categoria	Requisito
RNF01	Segurança	Senhas com **BCrypt**; autenticação **JWT** stateless; a API nunca devolve a senha/hash.
RNF02	Segurança	O identificador do usuário vem do **token**, nunca de parâmetro de URL (`?usuario=`). Cada usuário só acessa os próprios dados.
RNF03	Segurança	Entrada validada (Bean Validation) e saída escapada no frontend (sem `innerHTML` com dado do usuário).
RNF04	Segurança	Segredos (senha do banco, chave JWT) por **variáveis de ambiente**, nunca no repositório. CORS restrito à origem do frontend.
RNF05	Desempenho	Listagens respondem em menos de 2 s para 1.000 transações do usuário.
RNF06	Usabilidade	Fluxo “lançar despesa” em no máximo 4 interações; mensagens de erro claras em português.
RNF07	Responsividade	Layout **mobile-first**, funcional a partir de 360 px de largura até desktop.
RNF08	Acessibilidade	Contraste mínimo AA, campos com `label` , navegação por teclado, foco visível.
RNF09	Manutenibilidade	Camadas Controller → Service → Repository; **DTOs** na API; entidades não expostas.

RNF10	Manutenibilidade	Esquema do banco versionado por **Flyway**; `ddl-auto=validate`.
RNF11	Confiabilidade	Valores monetários em `BigDecimal` / `NUMERIC(12,2)`; erros tratados por `@ControllerAdvice` com formato único.
RNF12	Testabilidade	Testes unitários de serviços e BDD (Cucumber) chamando o código real; banco de teste isolado (Testcontainers).
RNF13	Portabilidade	`docker-compose` sobe banco (e opcionalmente backend); README com passo a passo.
RNF14	Compatibilidade	Chrome, Edge, Firefox e Safari atuais.
RNF15	Segurança	A exportação CSV usa apenas dados do usuário do token e neutraliza injeção de fórmulas (RN18).
RNF16	Acessibilidade	Os dois temas (claro e escuro) mantêm contraste AA; cores definidas só por variáveis CSS.
RNF17	Confiabilidade	A geração de recorrências é idempotente: rodar o job duas vezes não duplica transações (RN15).

3. Regras de negócio

ID	Regra
RN01	O valor de uma transação é sempre **> 0**; o sinal vem do tipo (receita soma, despesa subtrai).
RN02	O **tipo** (RECEITA/DESPESA) e o **grupo** (NECESSIDADES/LAZER/INVESTIMENTOS) da transação são **derivados da categoria**. O cliente envia apenas a categoria.
RN03	Receitas não pertencem a grupo de orçamento.
RN04	**Saldo = Σ receitas − Σ despesas.** Aportes em Investimentos contam como despesa (saem do saldo disponível).
RN05	Orçamento: os três percentuais são ≥ 0 e **somam exatamente 100%**. Padrão: 50 / 30 / 20.
RN06	**Limite do grupo = renda mensal × percentual.** Se a renda for 0, nenhum limite é calculado e o aviso não é exibido (o app pede para informar a renda).
RN07	Gasto do grupo = Σ despesas do mês/ano no grupo. **% usado = gasto ÷ limite.** Níveis: **OK** < 80%, **ATENÇÃO** de 80% a 100%, **ESTOURO** > 100%.
RN08	O alerta é informativo: **nunca bloqueia** o lançamento da despesa.
RN09	Meta: progresso = valor atual ÷ valor objetivo (exibido limitado a 100%). Concluída quando valor atual ≥ objetivo. Objetivo > 0; valor atual ≥ 0.
RN10	E-mail único; senha com no mínimo 8 caracteres; renda mensal ≥ 0.
RN11	Um usuário só lê, altera ou exclui dados que lhe pertencem (transações e metas).
RN12	Categorias são **fixas** (carregadas por migration) e iguais para todos os usuários.

Fórmulas da calculadora (RN13)

Seja `P` a aplicação inicial, `PMT` o aporte mensal (feito no **fim** de cada mês), `i` a taxa mensal em decimal (`taxa% ÷ 100`) e `n` o prazo em meses.

- **Composto:** `M = P·(1+i)^n + PMT · ((1+i)^n − 1) / i` (se `i = 0`, `M = P + PMT·n`)
- **Simples:** `M = P·(1 + i·n) + PMT·n + PMT·i·n·(n−1)/2`
- **Total investido** = `P + PMT·n`; **rendimento** = `M − total investido`.

Conferência (exemplo da V1): `P=0`, `PMT=300`, `i=1%`, `n=12`, composto → **R$ 3.804,75**.

Arredondamento: `HALF_EVEN` / `HALF_UP` com 2 casas apenas no resultado final.

Regras de recorrência, exportação e tema

ID	Regra
RN14	Só categorias do tipo **DESPESA** podem ser recorrentes. O dia do mês vai de 1 a 31; em meses mais curtos vale o **último dia** do mês.
RN15	Um job diário (`@Scheduled`) gera as ocorrências cuja data já chegou. Cada recorrência guarda a **última competência gerada** (mês); a geração avança mês a mês até o mês atual, inclusive meses perdidos, sem duplicar. Ao criar uma recorrência, as ocorrências já vencidas são geradas na hora.
RN16	Editar, pausar ou excluir uma recorrência afeta **somente meses futuros**. Transações já geradas permanecem, são transações comuns (editáveis e excluíveis) e continuam contando no orçamento. Excluir a recorrência mantém as transações geradas (vínculo vira nulo).
RN17	Recorrência só gera dentro do período (`data_inicio` ≤ ocorrência ≤ `data_fim` , se houver) e enquanto estiver **ativa**.
RN18	**CSV:** colunas `Data;Tipo;Grupo;Categoria;Descrição;Valor`; separador `;` ; valor com vírgula decimal e sem símbolo de moeda; datas `dd/MM/yyyy` ; codificação UTF-8 **com BOM** (abre certo no Excel). Textos iniciados por `=` , `+` , `-` ou `@` recebem apóstrofo na frente. Exporta só as transações do usuário no mês/ano pedido.
RN19	**Tema:** sem escolha do usuário, vale o `prefers-color-scheme` do sistema. A escolha manual é salva em `localStorage` e prevalece.

4. Categorias fixas

Tipo	Grupo	Categorias
RECEITA	—	Salário · Freelance · Rendimentos · Mesada/Presente · Reembolso · Outras receitas
DESPESA	NECESSIDADES	Moradia · Contas fixas · Alimentação · Transporte · Saúde · Educação · Parcelas e dívidas
DESPESA	LAZER	Delivery · Passeio · Assinaturas · Compras e vestuário · Viagem · Outros (lazer)
DESPESA	INVESTIMENTOS	Renda fixa · Renda variável · Reserva de emergência · Previdência

Exemplos de uso (campo **descrição** livre):

- *Delivery* → “pizza”
- *Contas fixas* → “internet”, “luz”
- *Assinaturas* → “streaming”

5. Rastreabilidade resumida

Funcionalidade	Requisitos	Regras
Cadastro/Login	RF01–RF04, RNF01–RNF04	RN10, RN11
Transações	RF05–RF10	RN01–RN04, RN12
Orçamento e alerta	RF11–RF14	RN05–RN08

Metas	RF15–RF17	RN09
Calculadora	RF18–RF19	RN13
Recorrências	RF20	RN14–RN17
Exportar CSV	RF21	RN18
Dark mode	RF22	RN19

03 · Metodologia de Desenvolvimento

1. Abordagem

Continuamos com **desenvolvimento ágil em sprints semanais** (Scrum leve), como na V1. O trabalho é incremental: cada sprint termina com algo que roda e pode ser demonstrado.

Elemento	Como fazemos
Duração da sprint	1 semana
Planejamento	No início: escolher itens do backlog (GitHub Projects) e definir a meta da sprint
Acompanhamento	Quadro Kanban: `Backlog → A fazer → Em andamento → Em revisão → Pronto`
Revisão/retrospectiva	No fim: demonstrar o incremento, listar o que funcionou e o que ajustar
Priorização	MoSCoW (ver `02-requisitos.md`)
Papéis	Todos desenvolvem; um membro rotativo facilita a sprint (planejamento e revisão)

2. Fluxo de trabalho com Git

- Branch principal protegida: `main` (sempre funcional).
- Branches curtas por tarefa: `feature/<resumo>`, `fix/<resumo>`, `docs/<resumo>`.
- **Pull Request obrigatório** para entrar na `main`, com pelo menos **1 revisão** de outro membro.
- **Commits convencionais** (em português, imperativo):
  - `feat: adiciona filtro de transações por mês`
  - `fix: corrige cálculo do saldo`
  - `docs: atualiza diagrama de classes`
  - `refactor:`, `test:`, `chore:`
- Cada tarefa do quadro vira uma *issue*; o PR referencia a issue (`Closes #12`).

3. Definition of Ready (pronto para começar)

Uma tarefa só entra na sprint se tiver: objetivo claro, requisitos/regras relacionados (IDs `RF` / `RN`), critério de aceite e estimativa (P/M/G).

4. Definition of Done (pronto para entregar)

- [ ] Código compila e a aplicação sobe localmente.
- [ ] Testes novos/ajustados passam (`./mvnw test`).
- [ ] Regras de negócio da tarefa cobertas por teste.
- [ ] Sem segredos, senhas ou dados reais no código.
- [ ] Revisado por outro membro (PR aprovado).
- [ ] **Documentação atualizada** (requisitos, API e diagramas Mermaid, se afetados).
- [ ] Interface verificada em celular (≈360 px) e desktop, quando houver tela.

5. Qualidade e testes

Nível	Ferramenta	O que cobre
Unitário	JUnit 5 + Mockito	Regras de negócio nos *services* (saldo, alerta, juros, metas)
Integração	Spring Boot Test + **Testcontainers (PostgreSQL)**	Repositórios, migrations e endpoints protegidos
Aceitação (BDD)	Cucumber (português)	Fluxos: cadastro/login, lançar despesa com alerta, meta, calculadora
Manual	Roteiro por tela	Responsividade, acessibilidade básica, bugs

		visuais

Regra: os passos do Cucumber **chamam o código real** (services/API). Nada de “login simulado” com senha fixa, como na V1.

6. Arquitetura e padrões

- **Backend:** MVC em camadas — `controller` (REST/DTO) → `service` (regras) → `repository` (Spring Data JPA) → `model` (entidades).
- **Segurança:** Spring Security, filtro JWT, `PasswordEncoder` BCrypt.
- **Erros:** `@RestControllerAdvice` com resposta padronizada (`status` , `mensagem` , `campos`).
- **Banco:** PostgreSQL com migrations **Flyway** (`db/migration/V1__...sql`).
- **Frontend:** HTML5 + CSS3 (variáveis CSS, mobile-first) + JavaScript modular (ES modules), sem framework.
- **Configuração:** `application.properties` lê variáveis de ambiente (`DB_URL` , `DB_USER` , `DB_PASSWORD` , `JWT_SECRET`).

7. Documentação como código

Toda a documentação vive em `docs/` em Markdown, e os diagramas em **Mermaid** (texto versionável). Assim o histórico do Git mostra quando o diagrama mudou e o Claude Code (ou qualquer colega) consegue ler o contexto sem abrir imagens.

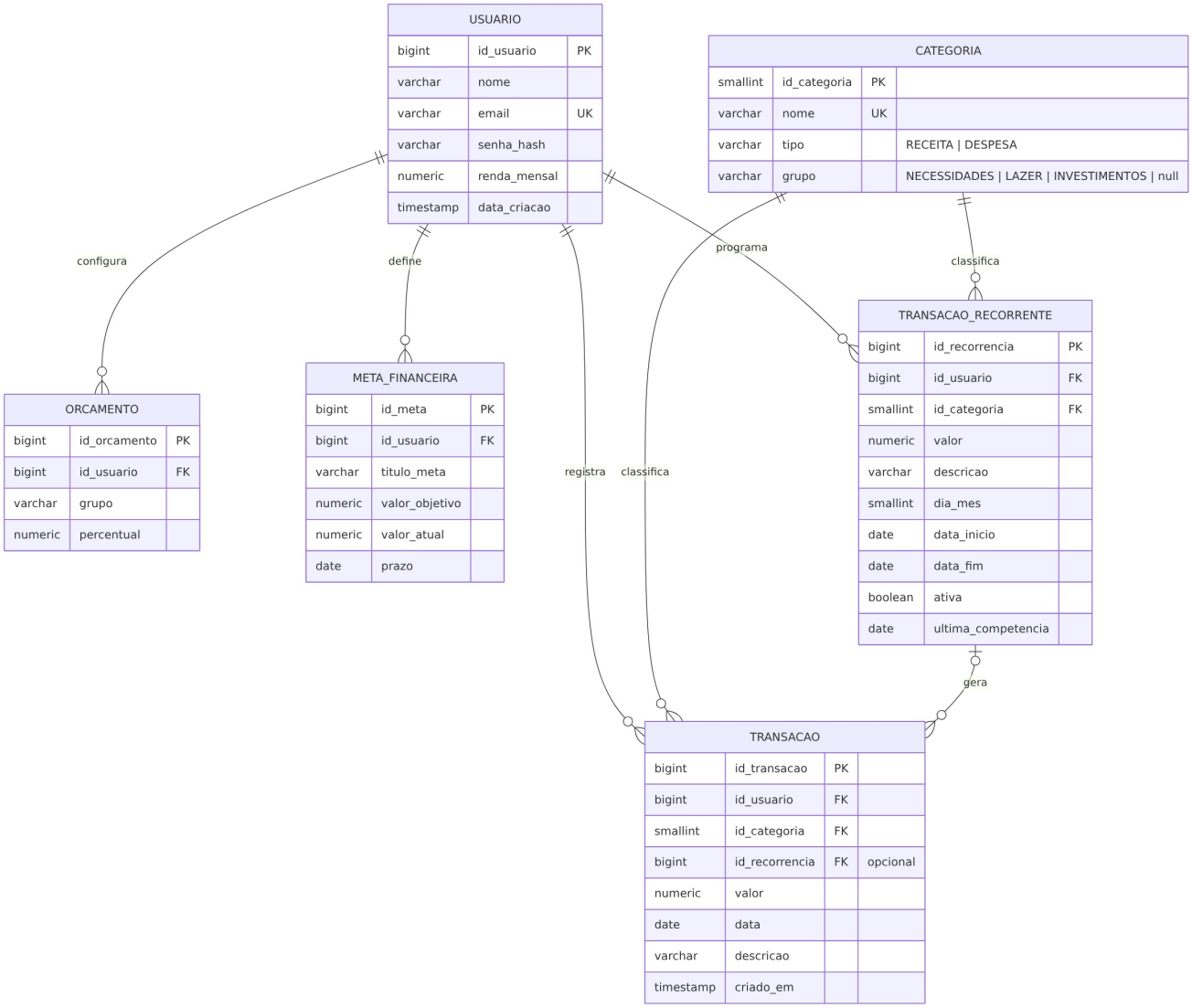
Arquivo	Conteúdo
`01-escopo.md`	Visão, escopo, riscos
`02-requisitos.md`	RF, RNF, regras de negócio, categorias
`03-metodologia.md`	Este documento
`04-modelo-de-dados.md`	DER, dicionário e SQL alvo
`05-diagramas.md`	Casos de uso, classes, sequência, BPMN, arquitetura
`06-api.md`	Contrato dos endpoints
`07-roadmap.md`	Sprints, backlog e decisões (ADRs)
`08-diagnostico-v1.md`	Problemas encontrados na V1

04 · Modelo de Dados

1. Decisões do modelo

- **Sai:** `conta_carteira` e `investimento` (a calculadora não persiste nada).
- **Categoria é fixa e global** (sem `id_usuario`), carregada por migration. Ela carrega o **tipo** e o **grupo**.
- **Transação não guarda o tipo nem o grupo**: ambos vêm da categoria (evita inconsistência, regra RN02).
- **Orçamento** é a configuração de 3 percentuais por usuário (um por grupo). O limite em R$ é calculado: `renda × percentual`.
- **Recorrência** é um modelo (`transacao_recorrente`) que gera transações comuns; a transação gerada guarda `id_recorrencia` (opcional) e depois vive sozinha.
- Valores monetários: `NUMERIC(12,2)`. Percentuais: `NUMERIC(5,2)`.
- Esquema criado **somente por migrations Flyway**; JPA com `ddl-auto=validate`.

2. Diagrama entidade-relacionamento



3. Dicionário de dados

usuario

Coluna	Tipo	Regras
id_usuario	BIGSERIAL	PK
nome	VARCHAR(100)	obrigatório

email	VARCHAR(150)	obrigatório, **único**
senha_hash	VARCHAR(100)	hash BCrypt (nunca a senha)
renda_mensal	NUMERIC(12,2)	≥ 0, padrão 0
data_criacao	TIMESTAMP	padrão `now()`

categoria (fixa)

Coluna	Tipo	Regras
id_categoria	SMALLSERIAL	PK
nome	VARCHAR(50)	obrigatório, único
tipo	VARCHAR(10)	`RECEITA` ou `DESPESA`
grupo	VARCHAR(15)	`NULL` para receita; `NECESSIDADES`, `LAZER` ou `INVESTIMENTOS` para despesa

transacao

Coluna	Tipo	Regras
id_transacao	BIGSERIAL	PK
id_usuario	BIGINT	FK → usuario (CASCADE)
id_categoria	SMALLINT	FK → categoria (RESTRICT)
id_recorrencia	BIGINT	FK → transacao_recorrente (SET NULL), opcional
valor	NUMERIC(12,2)	**> 0**
data	DATE	obrigatório
descricao	VARCHAR(255)	texto livre (ex.: “pizza”, “internet”)
criado_em	TIMESTAMP	padrão `now()`

orcamento

Coluna	Tipo	Regras
id_orcamento	BIGSERIAL	PK
id_usuario	BIGINT	FK → usuario (CASCADE)
grupo	VARCHAR(15)	`NECESSIDADES`, `LAZER`, `INVESTIMENTOS`
percentual	NUMERIC(5,2)	entre 0 e 100; **soma por usuário = 100** (validada no service)
—	—	`UNIQUE (id_usuario, grupo)`

meta_financeira

Coluna	Tipo	Regras
id_meta	BIGSERIAL	PK
id_usuario	BIGINT	FK → usuario (CASCADE)
titulo_meta	VARCHAR(100)	obrigatório
valor_objetivo	NUMERIC(12,2)	> 0
valor_atual	NUMERIC(12,2)	≥ 0, padrão 0
prazo	DATE	opcional

transacao_recorrente

Coluna	Tipo	Regras
id_recorrencia	BIGSERIAL	PK

id_usuario	BIGINT	FK → usuario (CASCADE)
id_categoria	SMALLINT	FK → categoria (RESTRICT); precisa ser de tipo DESPESA (validado no service)
valor	NUMERIC(12,2)	> 0
descricao	VARCHAR(255)	texto livre (ex.: “internet”)
dia_mes	SMALLINT	1 a 31
data_inicio	DATE	obrigatória
data_fim	DATE	opcional; ≥ data_inicio
ativa	BOOLEAN	padrão `true`
ultima_competencia	DATE	1º dia do último mês já gerado; `NULL` se nunca gerou

4. Migration alvo — `V1__criar_esquema.sql`

```
CREATE TABLE usuario (
    id_usuario    BIGSERIAL PRIMARY KEY,
    nome          VARCHAR(100)  NOT NULL,
    email         VARCHAR(150)  NOT NULL UNIQUE,
    senha_hash    VARCHAR(100)  NOT NULL,
    renda_mensal  NUMERIC(12,2) NOT NULL DEFAULT 0 CHECK (renda_mensal >= 0),
    data_criacao  TIMESTAMP     NOT NULL DEFAULT now()
);

CREATE TABLE categoria (
    id_categoria SMALLSERIAL PRIMARY KEY,
    nome         VARCHAR(50) NOT NULL UNIQUE,
    tipo         VARCHAR(10) NOT NULL CHECK (tipo IN ('RECEITA', 'DESPESA')),
    grupo        VARCHAR(15) CHECK (grupo IN ('NECESSIDADES', 'LAZER', 'INVESTIMENTOS')),
    CONSTRAINT ck_categoria_tipo_grupo CHECK (
        (tipo = 'RECEITA' AND grupo IS NULL) OR
        (tipo = 'DESPESA' AND grupo IS NOT NULL)
    )
);

CREATE TABLE transacao_recorrente (
    id_recorrencia     BIGSERIAL PRIMARY KEY,
    id_usuario         BIGINT        NOT NULL REFERENCES usuario(id_usuario) ON DELETE CASCADE,
    id_categoria       SMALLINT      NOT NULL REFERENCES categoria(id_categoria) ON DELETE RESTRICT,
    valor              NUMERIC(12,2) NOT NULL CHECK (valor > 0),
    descricao          VARCHAR(255),
    dia_mes            SMALLINT      NOT NULL CHECK (dia_mes BETWEEN 1 AND 31),
    data_inicio        DATE          NOT NULL,
    data_fim           DATE,
    ativa              BOOLEAN       NOT NULL DEFAULT TRUE,
    ultima_competencia DATE,
    CHECK (data_fim IS NULL OR data_fim >= data_inicio)
);
CREATE INDEX idx_recorrente_usuario ON transacao_recorrente (id_usuario);

CREATE TABLE transacao (
    id_transacao   BIGSERIAL PRIMARY KEY,
    id_usuario     BIGINT        NOT NULL REFERENCES usuario(id_usuario) ON DELETE CASCADE,
    id_categoria   SMALLINT      NOT NULL REFERENCES categoria(id_categoria) ON DELETE RESTRICT,
    id_recorrencia BIGINT        REFERENCES transacao_recorrente(id_recorrencia) ON DELETE SET NULL,
    valor          NUMERIC(12,2) NOT NULL CHECK (valor > 0),
    data         DATE          NOT NULL,
    descricao    VARCHAR(255),
    criado_em    TIMESTAMP     NOT NULL DEFAULT now()
);
CREATE INDEX idx_transacao_usuario_data ON transacao (id_usuario, data);

CREATE TABLE orcamento (
    id_orcamento BIGSERIAL PRIMARY KEY,
    id_usuario   BIGINT       NOT NULL REFERENCES usuario(id_usuario) ON DELETE CASCADE,
    grupo        VARCHAR(15)  NOT NULL CHECK (grupo IN ('NECESSIDADES', 'LAZER', 'INVESTIMENTOS')),
    percentual   NUMERIC(5,2) NOT NULL CHECK (percentual >= 0 AND percentual <= 100),
    UNIQUE (id_usuario, grupo)
);

CREATE TABLE meta_financeira (
```

```
    id_meta        BIGSERIAL PRIMARY KEY,
    id_usuario     BIGINT        NOT NULL REFERENCES usuario(id_usuario) ON DELETE CASCADE,
    titulo_meta    VARCHAR(100)  NOT NULL,
    valor_objetivo NUMERIC(12,2) NOT NULL CHECK (valor_objetivo > 0),
    valor_atual    NUMERIC(12,2) NOT NULL DEFAULT 0 CHECK (valor_atual >= 0),
    prazo          DATE
);
CREATE INDEX idx_meta_usuario ON meta_financeira (id_usuario);
```

5. Migration de dados — `V2__categorias_fixas.sql`

```
INSERT INTO categoria (nome, tipo, grupo) VALUES
  ('Salário',               'RECEITA', NULL),
  ('Freelance',             'RECEITA', NULL),
  ('Rendimentos',           'RECEITA', NULL),
  ('Mesada/Presente',       'RECEITA', NULL),
  ('Reembolso',             'RECEITA', NULL),
  ('Outras receitas',       'RECEITA', NULL),

  ('Moradia',               'DESPESA', 'NECESSIDADES'),
  ('Contas fixas',          'DESPESA', 'NECESSIDADES'),
  ('Alimentação',           'DESPESA', 'NECESSIDADES'),
  ('Transporte',            'DESPESA', 'NECESSIDADES'),
  ('Saúde',                 'DESPESA', 'NECESSIDADES'),
  ('Educação',              'DESPESA', 'NECESSIDADES'),
  ('Parcelas e dívidas',    'DESPESA', 'NECESSIDADES'),

  ('Delivery',              'DESPESA', 'LAZER'),
  ('Passeio',               'DESPESA', 'LAZER'),
  ('Assinaturas',           'DESPESA', 'LAZER'),
  ('Compras e vestuário',   'DESPESA', 'LAZER'),
  ('Viagem',                'DESPESA', 'LAZER'),
  ('Outros (lazer)',        'DESPESA', 'LAZER'),

  ('Renda fixa',            'DESPESA', 'INVESTIMENTOS'),
  ('Renda variável',        'DESPESA', 'INVESTIMENTOS'),
  ('Reserva de emergência', 'DESPESA', 'INVESTIMENTOS'),
  ('Previdência',           'DESPESA', 'INVESTIMENTOS');
```

> Dados de demonstração (usuário João com transações de exemplo) ficam em um script **separado** de desenvolvimento (`dev-seed.sql`), fora das migrations de produção. A senha de demonstração deve ser criada pela própria API de cadastro, para já sair com hash BCrypt.

6. Mapeamento JPA (orientação)

Entidade	Observações
`Usuario`	campo `senhaHash` ; **nunca** serializar para JSON
`Categoria`	`@Enumerated(STRING)` em `tipo` e `grupo` ; somente leitura
`Transacao`	`@ManyToOne Categoria` ; `usuarioId` vem do token; sem campos `tipo` / `grupo`
`Orcamento`	enum `GrupoOrcamento` ; um registro por (usuário, grupo)
`TransacaoRecorrente`	`@ManyToOne Categoria` ; `ultimaCompetencia` (`LocalDate`) controla a geração; `ativa` pausa o modelo
`MetaFinanceira`	`valorAtual` iniciando em `BigDecimal.ZERO`

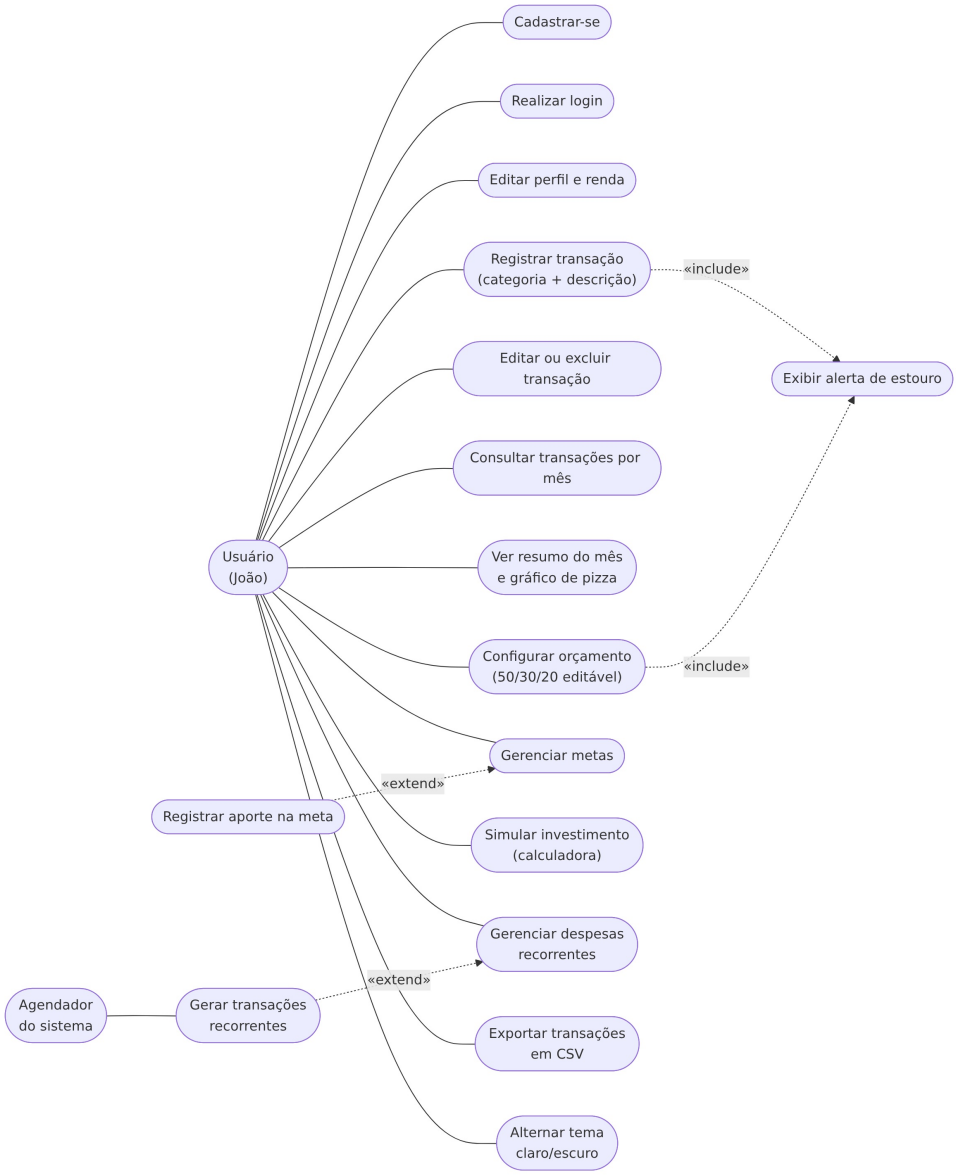
Enums Java: `TipoCategoria {RECEITA, DESPESA}` e `GrupoOrcamento {NECESSIDADES, LAZER, INVESTIMENTOS}`. Os enums antigos `TipoConta` , `TipoJuros` (persistido), `TipoOrcamento` e o `TipoCategoria` com 8 valores da V1 são removidos; `TipoJuros {SIMPLES, COMPOSTO}` continua existindo apenas como parâmetro da calculadora.

05 · Diagramas

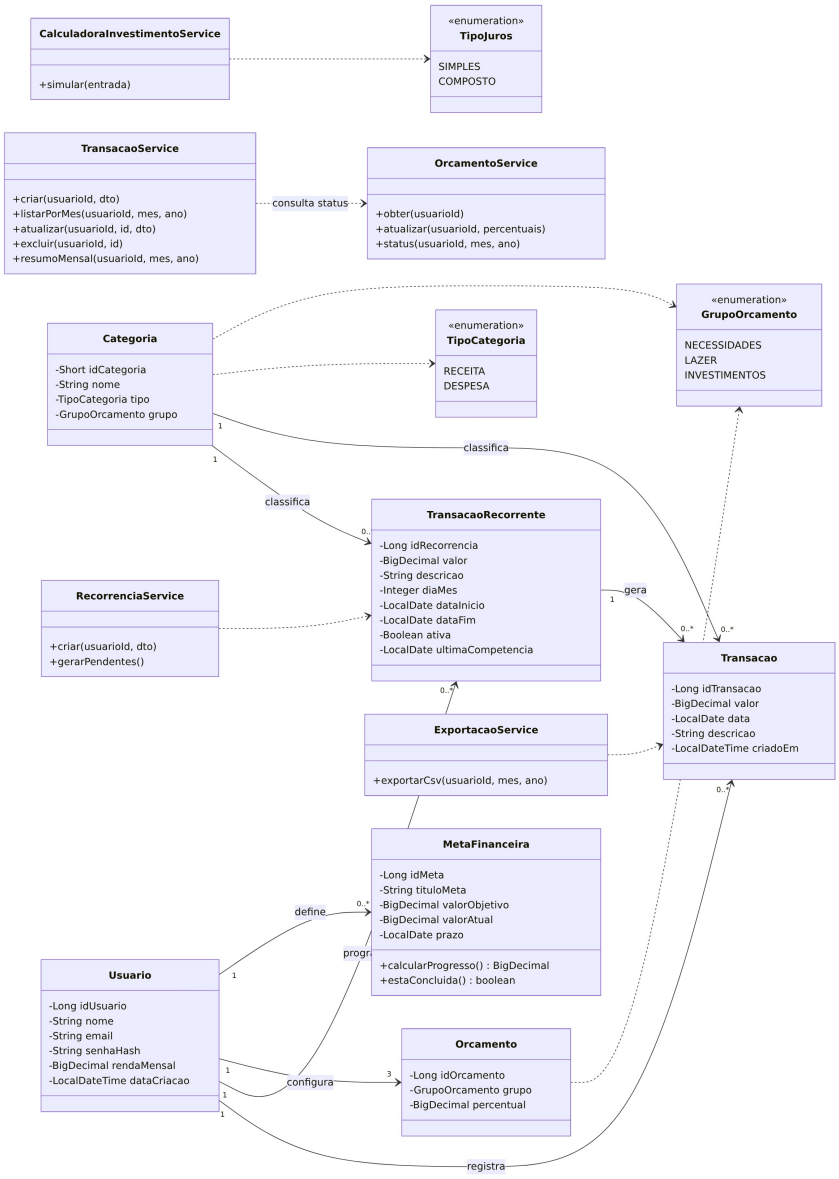
Todos os diagramas estão em **Mermaid** (renderizam direto no GitHub e no VS Code com extensão). O usuário de exemplo é **João**. Eles substituem os diagramas da V1 e refletem o escopo da V2 (sem Conta/Carteira, sem persistência de investimento).

1. Diagrama de casos de uso

Todos os casos de uso abaixo fazem parte do sistema **IFO**; o único ator é o usuário (João).



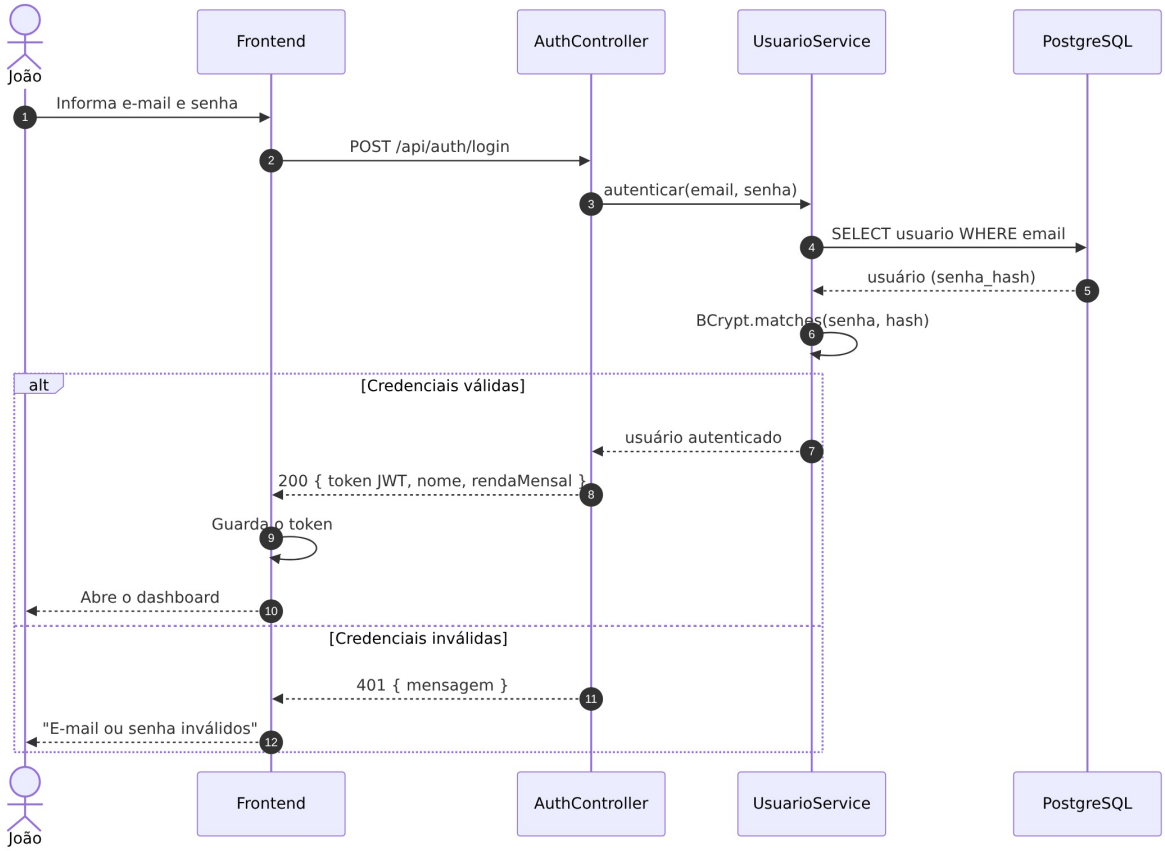
2. Diagrama de classes



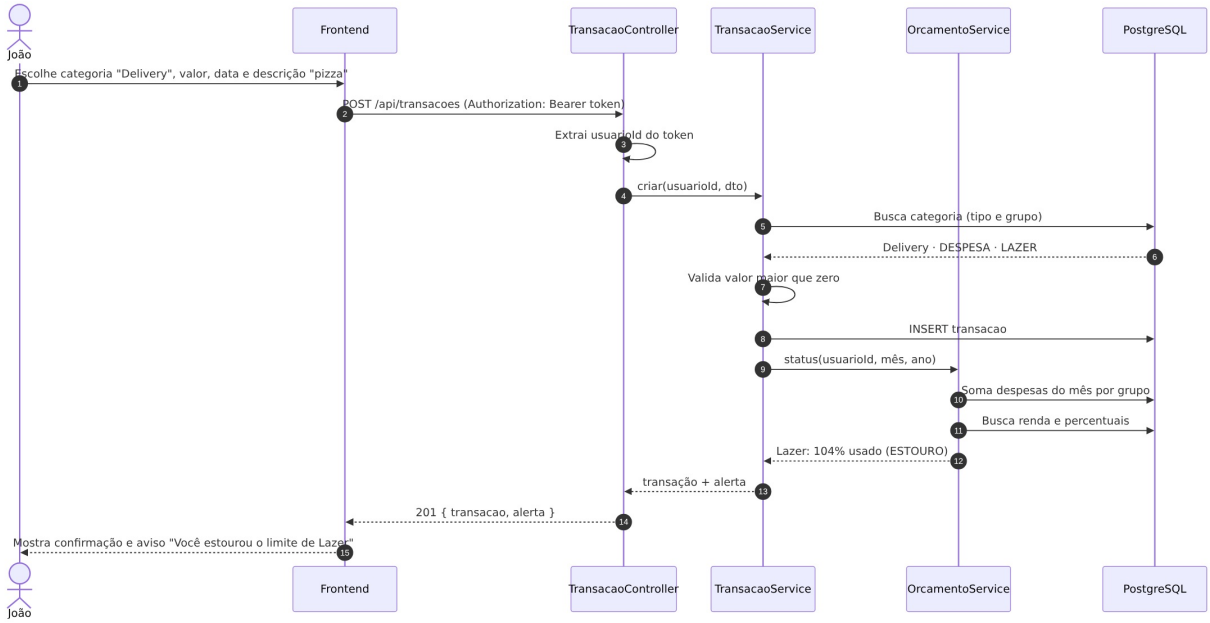
> `ResumoMensal`, `ResultadoSimulacao`, `StatusOrcamento` e `TransacaoResposta` são **DTOs** (não são tabelas). Não existe classe de conta/carteira nem de investimento persistido.

3. Diagramas de sequência

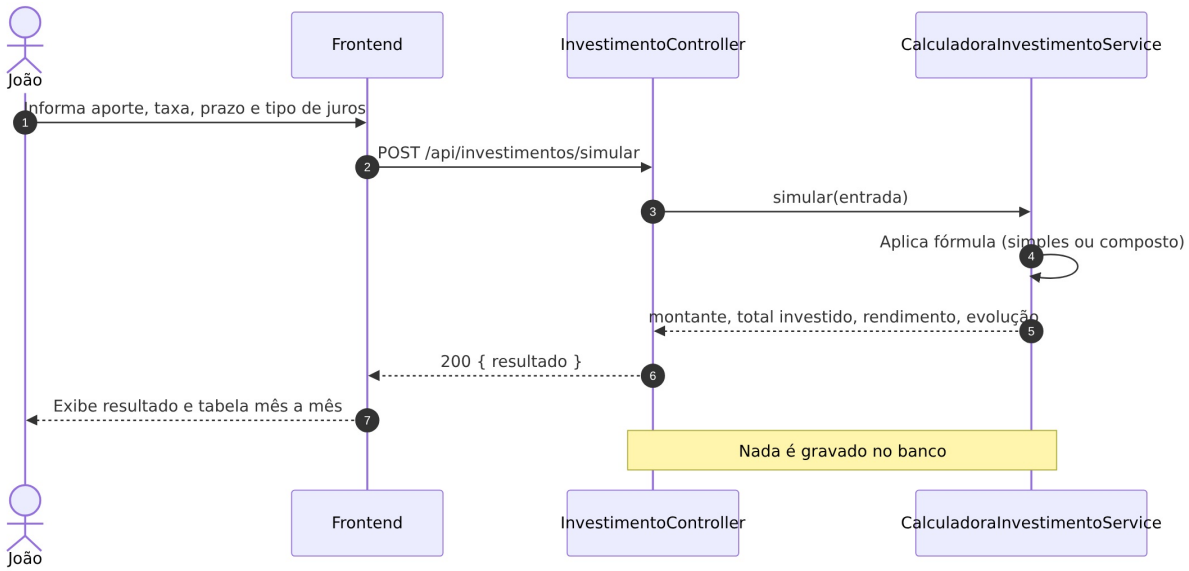
3.1 Login



3.2 Registrar despesa com alerta de orçamento



3.3 Simular investimento (sem persistência)

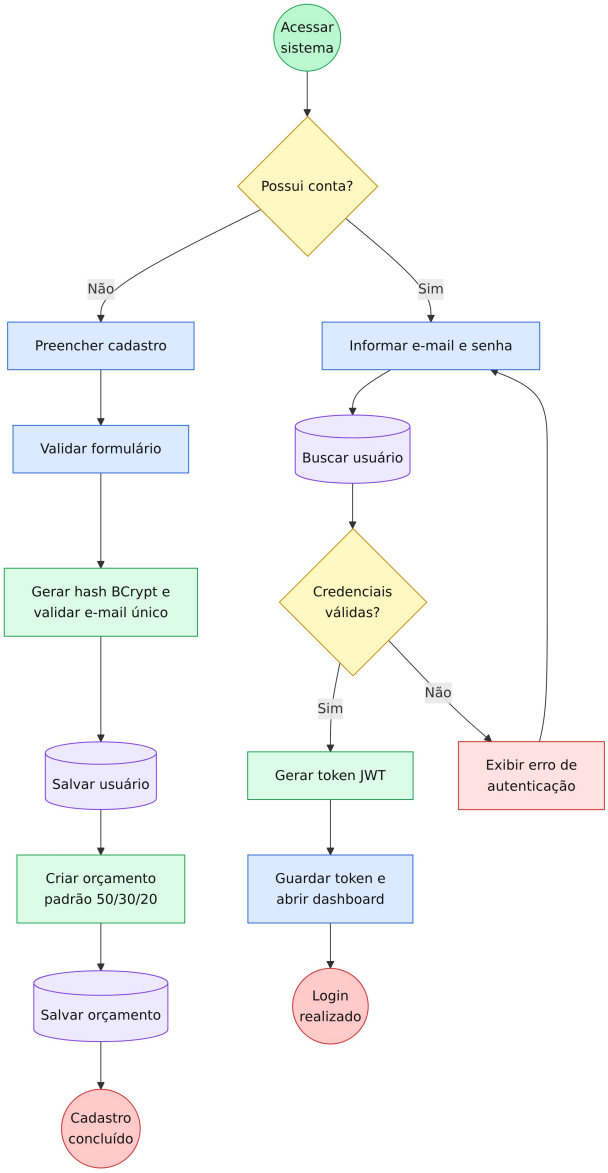


4. Diagramas de processo (BPMN simplificado)

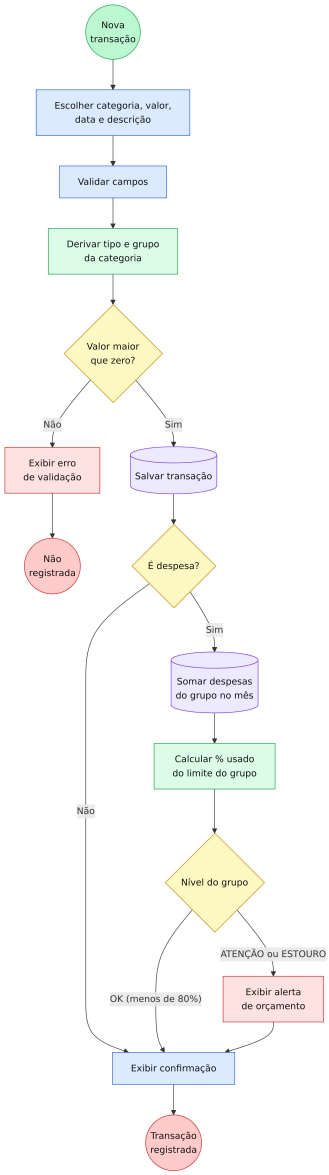
Os processos seguem a legenda da V1: **cor** indica quem executa a tarefa. Formas: retângulo = tarefa, losango = decisão, círculo = início/fim, cilindro = banco de dados.

Cor	Significado
Azul	Tarefa do usuário / frontend
Verde	Tarefa do backend (processamento)
Roxo	Persistência no banco de dados
Vermelho	Mensagem de alerta ou erro
Amarelo	Decisão (gateway exclusivo)

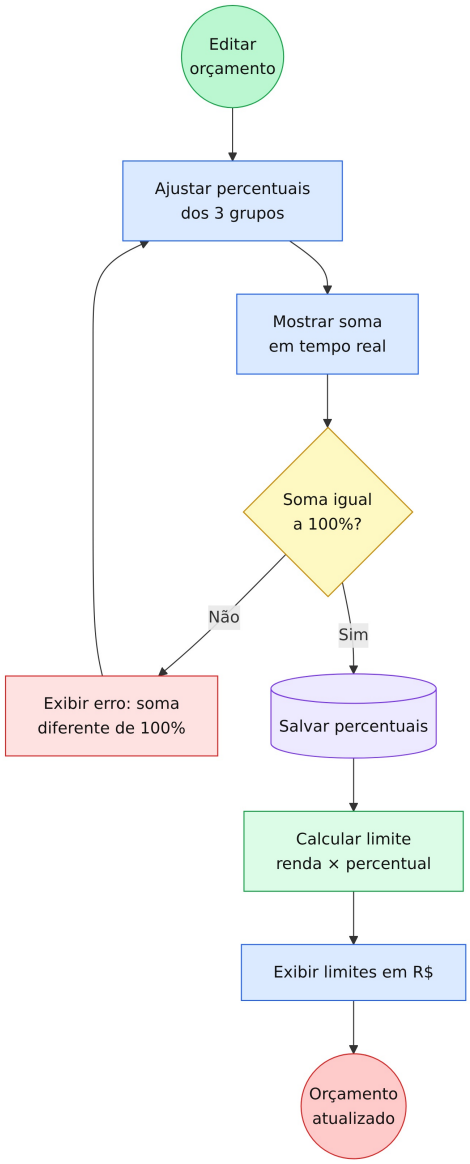
4.1 Processo 1 — Autenticação e cadastro



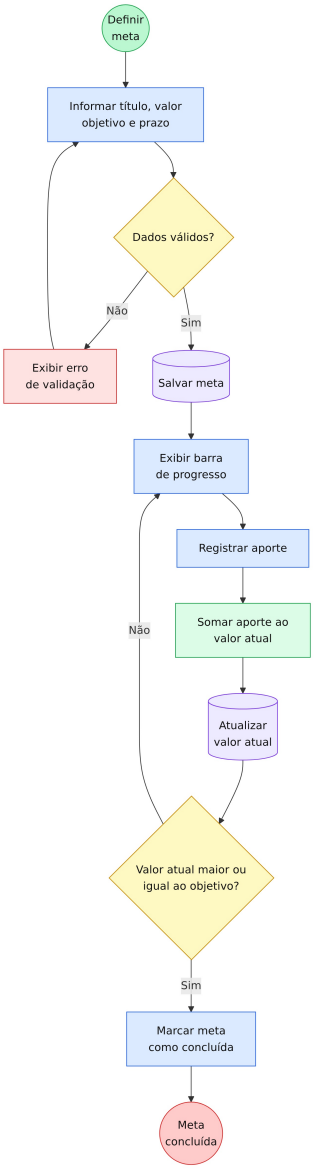
4.2 Processo 2 — Registrar transação e alertar estouro



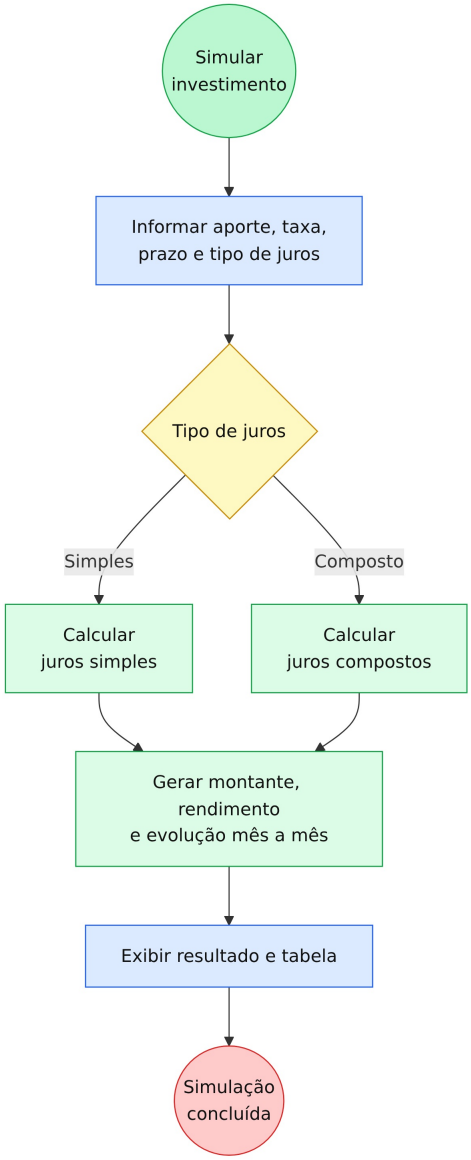
4.3 Processo 3 — Configurar orçamento



4.4 Processo 4 — Metas financeiras

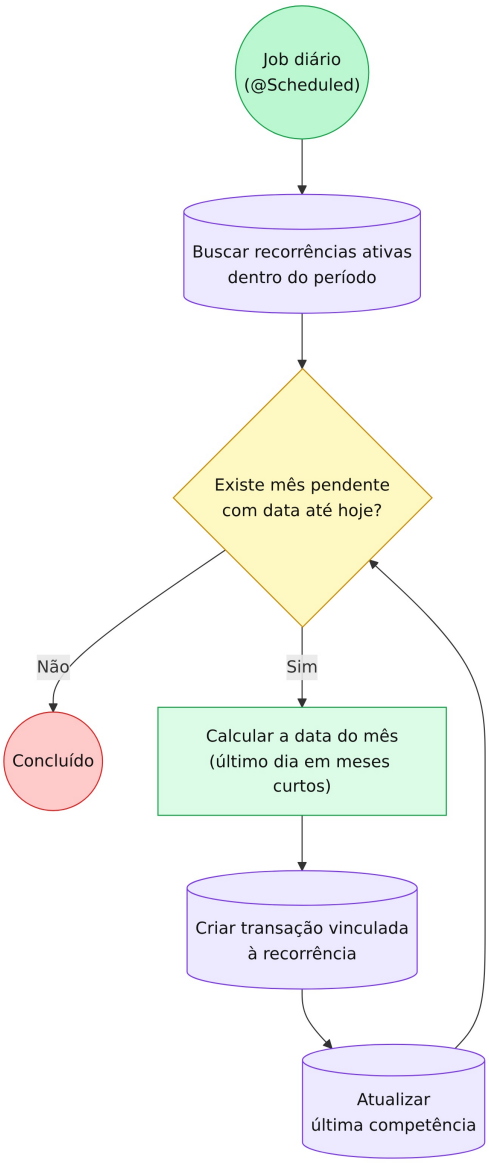


4.5 Processo 5 — Calculadora de investimentos



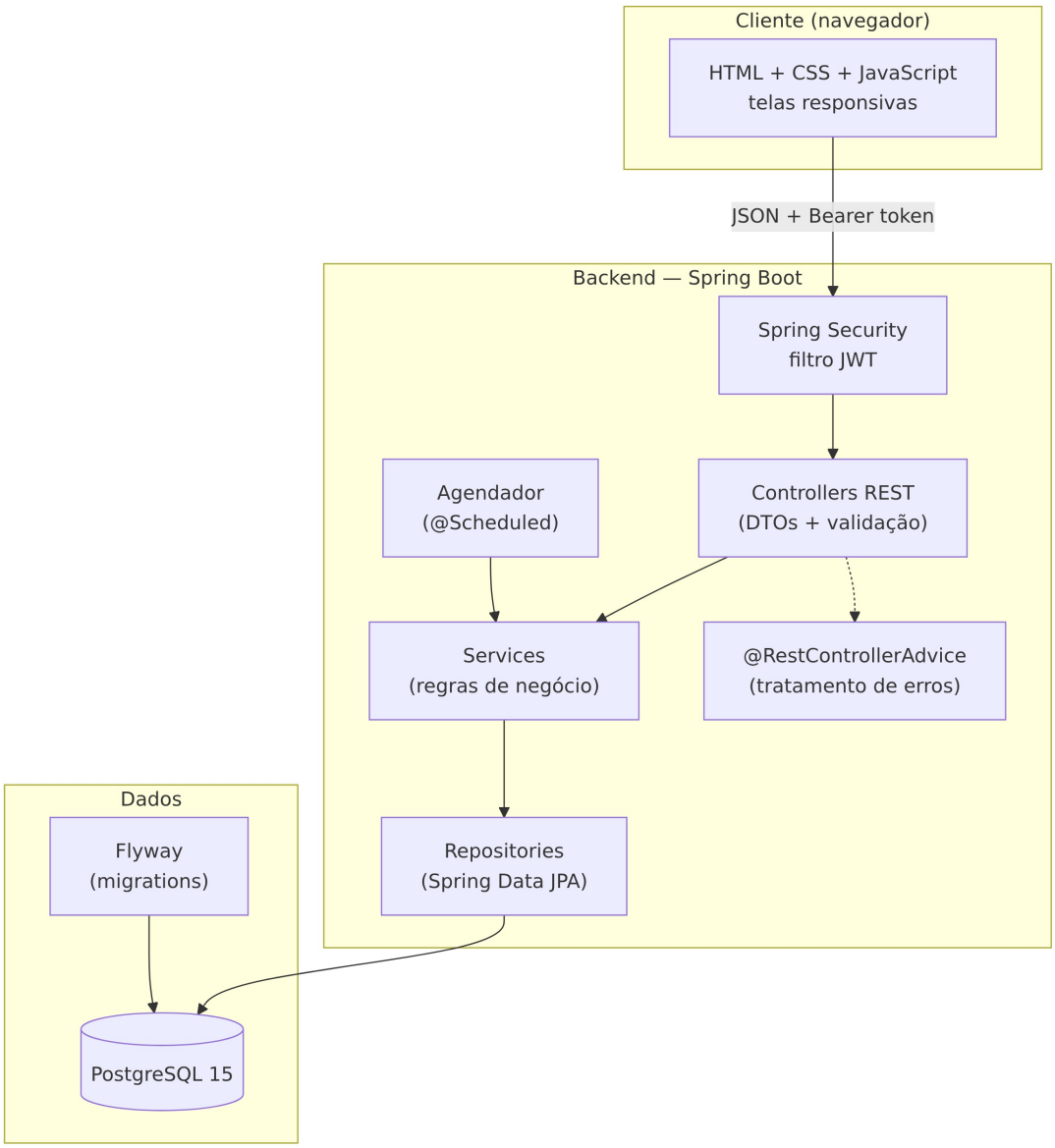
> Observação: a V1 tinha uma etapa “Salvar simulação no BD”. Ela foi **removida** (decisão ADR-003).

4.6 Processo 6 — Geração automática de despesa recorrente



> A geração é **idempotente**: a última competência impede duplicar transações mesmo que o job rode duas vezes ou fique dias parado (RN15).

5. Arquitetura em camadas



06 · Contrato da API (alvo V2)

Base: `http://localhost:8081/api` · Formato: JSON · Autenticação: `Authorization: Bearer <token>` (exceto `/auth/*`).

O `id` do usuário **nunca** vai na URL ou no corpo: o backend lê do token (RNF02). Entidades JPA **não** são expostas; tudo passa por DTOs.

1. Autenticação (público)

Método	Rota	Corpo	Resposta
POST	`/auth/cadastro`	`{ nome, email, senha, rendaMensal }`	`201 { token, usuario }`
POST	`/auth/login`	`{ email, senha }`	`200 { token, usuario }` · `401` se inválido

`usuario` = `{ idUsuario, nome, email, rendaMensal }` (sem senha/hash). O cadastro cria o orçamento padrão 50/30/20 (RF11).

2. Usuário

Método	Rota	Corpo	Resposta
GET	`/usuarios/me`	—	`200 usuario`
PUT	`/usuarios/me`	`{ nome, rendaMensal }`	`200 usuario`

3. Categorias (somente leitura)

Método	Rota	Resposta
GET	`/categorias`	`200 [ { idCategoria, nome, tipo, grupo } ]`

4. Transações

Método	Rota	Corpo / Query	Resposta
GET	`/transacoes?mes=10&ano=2026`	mês/ano opcionais (padrão: mês atual)	`200 [ transacao ]` ordenadas por data desc
POST	`/transacoes`	`{ idCategoria, valor, data, descricao }`	`201 { transacao, alerta }` (`alerta` é `null` se não houver)
PUT	`/transacoes/{id}`	mesmo corpo do POST	`200 { transacao, alerta }` · `404` se não for do usuário
DELETE	`/transacoes/{id}`	—	`204` · `404` se não for do usuário
GET	`/resumo?mes=10&ano=2026`	—	`200 resumo`

```
// transacao
{ "idTransacao": 12, "valor": 55.90, "data": "2026-10-17", "descricao": "pizza",
  "categoria": { "idCategoria": 14, "nome": "Delivery", "tipo": "DESPESA", "grupo": "LAZER" } }

// alerta (quando aplicável)
{ "grupo": "LAZER", "nivel": "ESTOURO", "percentualUsado": 104.2,
  "limite": 450.00, "gasto": 469.00,
  "mensagem": "Você ultrapassou o limite de Lazer deste mês." }

// resumo
{ "mes": 10, "ano": 2026, "totalReceitas": 1500.00, "totalDespesas": 1200.00, "saldo": 300.00,
  "despesasPorGrupo": [
    { "grupo": "NECESSIDADES", "total": 700.00, "percentual": 58.3 },
    { "grupo": "LAZER", "total": 300.00, "percentual": 25.0 },
    { "grupo": "INVESTIMENTOS", "total": 200.00, "percentual": 16.7 } ] }
```

Validações: `valor > 0`; `idCategoria` existente; `data` obrigatória; `descricao` até 255 caracteres.

5. Orçamento

Método	Rota	Corpo / Query	Resposta
GET	`/orcamento`	—	`200 [ { grupo, percentual } ]`
PUT	`/orcamento`	`{ "NECESSIDADES": 50, "LAZER": 30, "INVESTIMENTOS": 20 }`	`200` lista atualizada · `400` se a soma ≠ 100
GET	`/orcamento/status?mes=10&ano=2026`	—	`200 [ statusGrupo ]`

```
// statusGrupo
{ "grupo": "LAZER", "percentual": 30.0, "limite": 450.00, "gasto": 380.00,
  "percentualUsado": 84.4, "nivel": "ATENCAO" }
```

`nivel`: `OK` (< 80%), `ATENCAO` (80–100%), `ESTOURO` (> 100%). Se a renda for 0, `limite` vem `null` e `nivel` vem `SEM_LIMITE`.

6. Metas

Método	Rota	Corpo	Resposta
GET	`/metas`	—	`200 [ meta ]`
POST	`/metas`	`{ tituloMeta, valorObjetivo, prazo }`	`201 meta`
PUT	`/metas/{id}`	`{ tituloMeta, valorObjetivo, prazo }`	`200 meta`
DELETE	`/metas/{id}`	—	`204`
POST	`/metas/{id}/aportes`	`{ valor }` (> 0)	`200 meta`

```
{ "idMeta": 3, "tituloMeta": "Notebook", "valorObjetivo": 3000.00, "valorAtual": 450.00,
  "prazo": "2026-12-31", "progresso": 15.0, "concluida": false }
```

7. Calculadora de investimentos (sem persistência)

Método	Rota	Corpo	Resposta
POST	`/investimentos/simular`	ver abaixo	`200 resultado`

```
// entrada
{ "aplicacaoInicial": 0, "valorMensal": 300.00, "taxaMensal": 1.0, "tempoMeses": 12, "tipoJuros": "COMPOSTO" }

// resultado
{ "montanteFinal": 3804.75, "totalInvestido": 3600.00, "rendimento": 204.75,
  "evolucao": [ { "mes": 1, "investido": 300.00, "montante": 300.00 }, "..." ] }
```

Fórmulas em `02-requisitos.md` (RN13). Caso de teste de referência: entrada acima → **3804,75**.

8. Formato de erro (todas as rotas)

```
{ "timestamp": "2026-10-18T12:00:00", "status": 400, "erro": "Requisição inválida",
  "mensagem": "A soma dos percentuais deve ser 100.",
  "campos": { "percentual": "obrigatório" } }
```

Status	Quando
400	Validação falhou, regra de negócio violada
401	Token ausente, inválido ou expirado; login incorreto

404	Recurso não existe **ou não pertence ao usuário**
409	E-mail já cadastrado
500	Erro inesperado (mensagem genérica, sem stack trace)

9. Despesas recorrentes

Método	Rota	Corpo	Resposta
GET	`/recorrencias`	—	`200 [ recorrencia ]`
POST	`/recorrencias`	`{ idCategoria, valor, descricao, diaMes, dataInicio, dataFim }`	`201 recorrencia` (já gera as ocorrências vencidas)
PUT	`/recorrencias/{id}`	mesmo corpo do POST	`200 recorrencia` (afeta só meses futuros)
PATCH	`/recorrencias/{id}/ativa`	`{ "ativa": false }`	`200 recorrencia`
DELETE	`/recorrencias/{id}`	—	`204` (transações já geradas permanecem)


```
{ "idRecorrencia": 5, "valor": 99.90, "descricao": "internet", "diaMes": 10,
  "dataInicio": "2026-10-01", "dataFim": null, "ativa": true, "ultimaCompetencia": "2026-10-01",
  "categoria": { "idCategoria": 8, "nome": "Contas fixas", "tipo": "DESPESA", "grupo": "NECESSIDADES" } }
```


Validações: categoria existente e de tipo `DESPESA`; `valor > 0`; `diaMes` de 1 a 31; `dataFim` ≥ `dataInicio`. O job diário (`@Scheduled`) segue as regras RN14–RN17.

10. Exportação CSV

Método	Rota	Resposta
GET	`/transacoes/exportar?mes=10&ano=2026`	`200 text/csv; charset=UTF-8` com `Content-Disposition: attachment; filename="ifo-transacoes-2026-10.csv"`

```
Data;Tipo;Grupo;Categoria;Descrição;Valor
17/10/2026;DESPESA;LAZER;Delivery;pizza;55,90
10/10/2026;DESPESA;NECESSIDADES;Contas fixas;internet;99,90
05/10/2026;RECEITA;;Salário;pagamento;1500,00
```

O arquivo começa com BOM UTF-8. O frontend baixa com `fetch` enviando o token e cria o arquivo a partir do `Blob`. Regras em RN18.

> O **tema claro/escuro** (RF22) é só frontend: não existe endpoint.

07 · Roadmap, Backlog e Decisões

1. Visão geral das sprints

A V1 teve as Sprints 0–4. A refatoração continua a numeração com o prefixo **R** (R0–R5), uma semana cada.

Sprint	Meta	Entregas principais
R0 — Documentação	Contexto fechado e versionado	`docs/`, diagramas Mermaid, `CLAUDE.md`, README, PDF do seminário atualizado
R1 — Fundação	Backend alinhado ao banco e seguro	Flyway (V1/V2), entidades novas, DTOs, BCrypt + JWT, `@RestControllerAdvice`, variáveis de ambiente, `docker-compose`, remoção de Conta/Investimento
R2 — Núcleo financeiro	Transações de ponta a ponta	Categorias fixas, CRUD de transações (PUT), filtro por mês, resumo mensal
R3 — Orçamento, metas, calculadora e recorrências	Regras de negócio completas	Orçamento editável, status e alerta, metas com aportes, calculadora (juros simples/compostos), despesas recorrentes com job diário, exportação CSV
R4 — Frontend	Interface nova e responsiva	Design system (tokens CSS), mobile-first, telas refeitas, pizza simples, tema claro/escuro, telas de recorrências, botão exportar CSV, correção de bugs visuais, escape de HTML
R5 — Qualidade e entrega	Pronto para apresentar	Testes (unitário, integração, Cucumber), revisão de segurança, README final, vídeo/slides

> As sprints R3 e R4 podem andar em paralelo por pessoas diferentes depois que a API de R2 estiver estável.

2. Backlog detalhado

R1 — Fundação

- [ ] Criar `docker-compose.yml` com PostgreSQL 15 (credenciais por `.env`).
- [ ] Adicionar Flyway; criar `V1__criar_esquema.sql` e `V2__categorias_fixas.sql` (ver `04-modelo-de-dados.md`; inclui a tabela `transacao_recorrente`).
- [ ] Trocar `ddl-auto=none` por `validate`; mover segredos para variáveis de ambiente.
- [ ] Reescrever entidades: `Usuario` (senhaHash), `Categoria` (nome, tipo, grupo), `Transacao` (ManyToOne), `Orcamento`, `MetaFinanceira`.
- [ ] Remover: `ContaCarteira`, `Investimento` (entidade/repo/service/controller), `TipoConta`, `TipoOrcamento`, enum antigo `TipoCategoria`.
- [ ] Spring Security + filtro JWT + BCrypt; `AuthController` (`/auth/cadastro`, `/auth/login`).
- [ ] DTOs de entrada/saída; nenhuma entidade na resposta.
- [ ] `@RestControllerAdvice` com formato de erro padrão (`06-api.md` §8).
- [ ] CORS restrito à origem do frontend.

R2 — Núcleo financeiro

- [ ] `GET /categorias`.
- [ ] `TransacaoService`: criar/atualizar/excluir/listar por mês, sempre filtrando pelo usuário do token.
- [ ] Derivar tipo e grupo da categoria (RN02); validar valor > 0.
- [ ] `GET /resumo` (receitas, despesas, saldo, despesas por grupo).
- [ ] Testes unitários do saldo e do resumo.

R3 — Orçamento, metas, calculadora e recorrências

- [ ] Criar orçamento padrão 50/30/20 no cadastro.

- [ ] `GET/PUT /orcamento` com validação de soma = 100.
- [ ] `GET /orcamento/status` e alerta embutido na resposta de `POST/PUT /transacoes`.
- [ ] CRUD de metas + `POST /metas/{id}/aportes` + `progresso` e `concluida`.
- [ ] `POST /investimentos/simular` com `BigDecimal` (caso de referência 3804,75).
- [ ] Testes: níveis 79,9% / 80% / 100% / 100,01%; renda zero; soma ≠ 100.
- [ ] CRUD de recorrências (`/recorrencias`) com validação de categoria DESPESA (RN14).
- [ ] Job diário `@Scheduled` + geração ao criar a recorrência; avanço por `ultima_competencia`, sem duplicar (RN15–RN17).
- [ ] Testes: dia 31 em mês curto, mês perdido (catch-up), rodar o job duas vezes, recorrência pausada/encerrada.
- [ ] `GET /transacoes/exportar` (CSV `;`, BOM UTF-8, vírgula decimal, neutralização de `=+-@`) (RN18).

R4 — Frontend

- [ ] Definir design tokens (cores, tipografia, espaçamento, raios) em variáveis CSS.
- [ ] Layout **mobile-first** (360 px) com breakpoints para tablet e desktop; menu adaptado para celular.
- [ ] Reescrever `app.js` em módulos ES (`api.js`, `auth.js`, `ui.js` e um arquivo por tela).
- [ ] Cliente de API com token e tratamento central de erros (401 volta ao login).
- [ ] Telas: login, cadastro, dashboard, transações (filtro de mês), orçamento, metas, calculadora, perfil.
- [ ] Formulário de transação: **categoria → (grupo exibido automaticamente) → descrição**.
- [ ] Alerta de orçamento (faixa amarela/vermelha) no lançamento e no dashboard.
- [ ] Gráfico de pizza simples (SVG ou `conic-gradient`, sem biblioteca).
- [ ] Tela de **despesas recorrentes** (listar, criar, editar, pausar, excluir).
- [ ] Botão **Exportar CSV** na tela de transações, respeitando o filtro de mês.
- [ ] **Dark mode:** tokens de cor em `:root` e `[data-theme="dark"]`, padrão por `prefers-color-scheme`, botão de alternar, escolha em `localStorage`; conferir contraste AA nos dois temas e as cores do gráfico de pizza.
- [ ] Trocar `innerHTML` com dados do usuário por `textContent`/escape.
- [ ] Levantar e corrigir bugs visuais (checklist por tela abaixo).
- [ ] Acessibilidade básica: `label`, foco visível, contraste AA, teclado.

R5 — Qualidade e entrega

- [ ] Testcontainers (PostgreSQL) para integração.
- [ ] Reescrever Cucumber chamando o código real (login, transação com alerta, meta, calculadora).
- [ ] Revisão de segurança: ninguém acessa dado alheio; segredos fora do repo.
- [ ] README final, `dev-seed.sql`, passo a passo de execução.
- [ ] Atualizar o PDF do seminário com os diagramas novos.

3. Checklist de bugs visuais (preencher na R4)

Marcar por tela, em celular (360 px) e desktop (1280 px):

Tela	Sobreposição	Texto cortado	Menu/overflow	Estados vazios	Erros visíveis	Foco/contraste
Login / Cadastro	☐	☐	☐	—	☐	☐
Dashboard	☐	☐	☐	☐	☐	☐
Transações	☐	☐	☐	☐	☐	☐
Orçamento	☐	☐	☐	☐	☐	☐
Metas	☐	☐	☐	☐	☐	☐
Calculadora	☐	☐	☐	—	☐	☐

4. Backlog IFO V2 (futuro, fora deste ciclo)

- Dashboards interativos (gráficos dinâmicos, cartões, comparativos mês a mês).
- Integração com APIs bancárias (importação de transações).
- Aplicativo mobile nativo/híbrido.
- Autenticação de dois fatores.
- Sugestões de economia com IA.

5. Decisões de arquitetura (ADRs)

ID	Decisão	Motivo

ADR-001	Categorias **fixas**, em tabela carregada por migration (não enum)	Permite FK, grupo por categoria e evolução sem recompilar; usuário só edita a descrição
ADR-002	Tipo e grupo da transação **derivados da categoria**	Elimina inconsistência (ex.: “Delivery” marcado como Necessidade)
ADR-003	Calculadora **sem persistência**, calculada no backend	Escopo pedido: só calculadora; backend com `BigDecimal` é testável e fonte única da regra
ADR-004	**BCrypt + JWT** stateless	Resolve senha em texto puro e remove `?usuario=ID`
ADR-005	**Flyway** + `ddl-auto=validate`	Esquema reproduzível; fim da divergência entre SQL manual e entidades
ADR-006	Frontend em **HTML/CSS/JS puro** com módulos ES; pizza sem biblioteca	Mantém decisão da V1 (leveza, controle)
ADR-007	**Remover Conta/Carteira**	Não será usada; simplifica modelo e diagramas
ADR-008	Alerta **informativo**, limiar de atenção em 80%, nunca bloqueia	Pedido: “notificação simples de estouro”
ADR-009	Dashboards interativos **adiados** para o IFO V2	Controle de escopo
ADR-010	Diagramas em **Mermaid** versionados em `docs/`	Documentação sincronizada com o código e legível pelo Claude Code
ADR-011	**Despesas recorrentes** como modelo + job diário idempotente (`ultima_competencia`), só para DESPESA	Aprovado em 18/10/2026; evita duplicar, recupera meses perdidos e mantém a transação gerada como transação comum
ADR-012	**Exportar CSV** com `;`, vírgula decimal e UTF-8 com BOM	Abre direto no Excel em português; neutraliza injeção de fórmulas
ADR-013	**Dark mode** por variáveis CSS, `prefers-color-scheme` e `localStorage`	Sem biblioteca; reaproveita os design tokens da R4

Novas decisões entram nesta tabela com número sequencial.

08 · Diagnóstico da V1

Resultado da leitura do repositório `jmarcosvm/IFO-Organizador-Financeiro` (commit “Entrega final IFO”) e do PDF do seminário. Serve de ponto de partida da refatoração. Cada item aponta para a sprint em que será resolvido.

1. Segurança

#	Achado	Onde	Sprint
S1	**Senha em texto puro**: o login compara `String.equals` e o cadastro grava a senha como veio.	`UsuarioService`	R1
S2	`POST /usuarios` e `/usuarios/login` **devolvem o `Usuario` inteiro, com a senha**.	`UsuarioController`	R1
S3	**Sem autenticação/autorização**: qualquer pessoa lê, cria ou apaga dados de outro usuário mudando `?usuario=ID` ou o `idUsuario` do corpo.	todos os controllers	R1
S4	O PDF e o BPMN citam **BCrypt e JWT**, mas o `pom.xml` não tem Spring Security.	`pom.xml`	R1
S5	`@CrossOrigin(origins = "*")` em todos os controllers.	controllers	R1
S6	Credenciais do banco (`postgres` / `password`) no repositório público.	`application.properties`	R1
S7	**XSS**: `innerHTML` com `descricao` e outros campos sem escape.	`app.js`	R4
S8	Sem validação de entrada (`@Valid`), aceitando valor negativo/zero e campos nulos.	controllers/models	R1–R2
S9	Usuário “logado” é um objeto em `localStorage` , sem token.	`app.js`	R1/R4

2. Banco de dados e modelo

#	Achado	Sprint
B1	`Categoria` (entidade) usa `tipo` enum único + `descricao` ; a tabela tem `nome_categoria` , `id_usuario` e `tipo` RECEITA/DESPESA. **Não batem.**	R1
B2	`OrcamentoMensal` usa `mes` e `ano` ; a tabela mantém `mes_referencia NOT NULL` e não tem coluna `mes` . O insert deve falhar.	R1
B3	O script SQL termina com `ALTER COLUMN categoria DROP NOT NULL` **depois** de dropar a coluna `categoria` (erro), e tem `DELETE FROM orcamento_mensal` solto.	R1
B4	`ddl-auto=none` + ajustes manuais por `ALTER TABLE` no meio do script: banco não é reproduzível.	R1
B5	Enums do banco convertidos para `VARCHAR` com `ALTER` após criar (resquício de	R1

	tentativa-e-erro).	
B6	`Transacao` tem `categoriaTipo` redundante e `idUsuario` / `idConta` / `idCategoria` como `Long` solto (sem relacionamento JPA).	R1
B7	Tabela e entidade de **Conta/Carteira** e de **Investimento** existem, mas não serão usadas.	R1
B8	Dados de teste (usuário “Douglas”) misturados ao esquema, com hash de senha inválido (`$2y$10$92IXUNpkjO0rOQ5byMi` , truncado).	R1/R5

3. Regras de negócio

#	Achado	Sprint
N1	A regra 50/30/20 é só `renda × %` no frontend; **não compara com os gastos** e não há alerta de estouro (previsto no BPMN).	R3
N2	Orçamento padrão é **criado pelo frontend** ao abrir a tela vazia (3 `POST` disparados pelo `app.js`), e não pelo backend no cadastro.	R3
N3	O usuário **não consegue ajustar os percentuais**.	R3
N4	A simulação de investimento é calculada só no JS; o backend apenas grava o `montanteFinal` recebido.	R3
N5	O BPMN prevê “verificar saldo suficiente” antes da despesa; não existe.	descartado (RN08: alerta não bloqueia)
N6	Categoria e grupo hoje são a mesma coisa (enum de 8 valores). Falta o modelo Grupo → Categoria → Descrição.	R2
N7	O saldo ignora filtro de período (soma tudo desde sempre).	R2
N8	Metas: existe `atualizarValor` no service, mas nenhum endpoint o expõe (sem aporte pelo backend).	R3

4. API e backend

#	Achado	Sprint
A1	Editar transação no frontend é **DELETE + POST**: perde o id, não é atômico.	R2
A2	`DELETE /transacoes/{id}` não verifica dono nem existência.	R2
A3	Exceções genéricas (`RuntimeException`) e `orElseThrow()` sem mensagem; sem tratamento central.	R1
A4	Entidades JPA expostas direto na API.	R1
A5	Injeção por `@Autowired` em campo (preferir construtor).	R1
A6	`ContaController` só lista e cria; faltam editar/excluir, e não há tela. (Será removido.)	R1
A7	`inicializarCategoriasPadrao` faz `findAll()` dentro de loop. (Será	R1

	substituído por migration.)	

5. Frontend

#	Achado	Sprint
F1	`app.js` com ~800 linhas, tudo global, misturando API, regra e HTML.	R4
F2	Dashboard calcula receitas/despesas **no navegador** (faz 2 requisições: saldo e lista completa) e sem filtro de mês.	R2/R4
F3	`orcamentos.html` tem dois elementos com o mesmo `id="lista-orcamentos"` (HTML inválido).	R4
F4	Nome “Douglas” fixo no HTML até o JS rodar; rodapé “Sprint 3”.	R4
F5	`API_URL` fixa em `localhost:8081`.	R4
F6	Design e responsividade a melhorar; **bugs visuais a levantar** (checklist em `07-roadmap.md §3`).	R4

6. Testes

#	Achado	Sprint
T1	Os steps de login **não chamam o `UsuarioService`**: comparam com a senha fixa `"123456"`. O teste passa sem testar nada.	R5
T2	Testes de transação gravam no banco local real (precisam do Postgres no ar) e deixam lixo.	R5
T3	`IfoBackendApplicationTests` e Cucumber dependem de banco externo.	R5
T4	`surefire` configurado para incluir `**/*Steps.java` (classes de steps não são testes).	R5

7. Divergências entre o PDF do seminário e o código

PDF / diagramas	Código atual	Ação
Hash BCrypt e token JWT no processo de autenticação	Nada disso existe	Implementar (R1)
Diagramas com Conta/Carteira, “Gerenciar Contas” e “Gerenciar Categorias (CRUD)”	Contas sem tela; categorias fixas	Remover do escopo e dos diagramas (feito em `05-diagramas.md`)
Alerta de estouro e “Verificar saldo” no BPMN	Inexistentes	Alerta em R3; verificação descartada
Simulação com “Salvar simulação no BD”	Grava no banco um `montanteFinal` calculado no JavaScript	Calculadora sem persistência
`TransacaoReceita` e `TransacaoDespesa` no diagrama de classes	Uma entidade `Transacao` só	Diagrama novo usa `Transacao` única
Usuário “Douglas” nos diagramas	—	Trocado por “João”
Referência “FAWLER, Martin”	—	Corrigir para **FOWLER**
Resultado “totalmente funcional” e “atende a todos os requisitos”	Vários requisitos parciais	Reescrever resultados após a V2